18. Interview Preparation

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 18 of 18 · Thirupathi Keloth · v2.0

Contents

18.0 How DevOps / Cloud interviews work

18.1 Resume, GitHub and LinkedIn

18.2 How to answer

18.3 Rapid-fire fundamentals with model answers

18.4 Scenario questions with model answers

18.5 System design for DevOps interviews

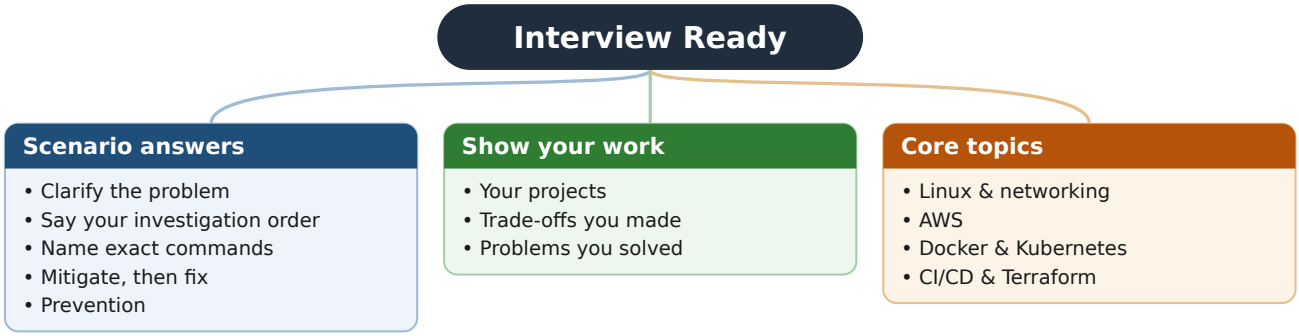
18.6 Hands-on tasks you may get (with solutions)

18.7 Behavioral questions (STAR)

18.8 Questions to ask the interviewer

18.9 Job search strategy

18.10 Final two-week revision plan



18.0 How DevOps / Cloud interviews work

Round	What they test	How to prepare
Recruiter screen	Fit, communication, salary range, notice period	60-second introduction, clear story of why DevOps
Technical screen	Breadth of fundamentals (Linux, networking, Git, cloud, containers, CI/CD)	Rapid-fire answers in 18.3
Hands-on / live task	Write a script, fix a Dockerfile/YAML, debug a broken environment, Terraform a resource	Labs from every topic; tasks in 18.6
Scenario / troubleshooting	Your method under ambiguity	Troubleshooting notes (17) and 18.4
System design	Architecture for availability, scale, security, cost	Framework and worked designs in 18.5
Behavioral / hiring manager	Ownership, collaboration, incidents, learning	STAR stories in 18.7

Interviewers look for: **solid fundamentals**, a **structured way of thinking**, **real hands-on experience** (details only a doer knows), awareness of **trade-offs**, security and cost, and honest communication.

18.1 Resume, GitHub and LinkedIn

Resume rules

- One or two pages; reverse chronological; a short summary line: *"DevOps engineer skilled in AWS, Terraform, Docker, Kubernetes and CI/CD, building automated, secure delivery pipelines."*
- Skills grouped: Cloud (AWS: EC2, VPC, IAM, S3, RDS, EKS) · IaC (Terraform, Ansible) · Containers (Docker, Kubernetes, Helm) · CI/CD (GitHub Actions, Jenkins, Argo CD) · Observability (Prometheus, Grafana, CloudWatch) · Scripting (Bash, Python) · OS (Linux).
- Every bullet = **action + technology + measurable result**:
- Weak: "Worked on CI/CD pipelines."
- Strong: "Built GitHub Actions pipelines with test, Trivy scanning and ECR push for 6 services, cutting release time from 2 hours to 12 minutes."
- Projects section (critical when you have no DevOps job yet): link each to a GitHub repo with a README, architecture diagram, and how to run it.
- Only list what you can defend in depth: every keyword invites questions.

Portfolio from this program

Project	Shows
Linux server automation (Bash + cron + logrotate)	Linux, scripting
Python automation (AWS boto3 reports, health checker with tests)	Python, AWS SDK, testing
AWS 3-tier architecture with Terraform (VPC, ALB, ASG, RDS)	Cloud architecture, IaC, cost awareness
Ansible role for web servers (idempotent, linted)	Configuration management
Dockerized app with Compose (multi-stage, non-root, healthchecks)	Containers
CI/CD pipeline (test, scan, build, push, deploy)	Delivery automation, DevSecOps
Kubernetes deployment with Helm/Kustomize + HPA + ingress	Kubernetes
Monitoring stack with SLO alerts	Observability, SRE
GitOps with Argo CD (dev → prod promotion)	Modern delivery
Final project: everything together on EKS	End-to-end ownership

Each README: problem, architecture diagram, tools, how to deploy and **destroy**, security choices, costs, what you'd improve. Never commit secrets (scan with gitleaks before making repos public).

LinkedIn

Headline with target role and stack, summary in first person, projects in Featured, posts about what you learned (short write-ups of labs and fixes build visibility). Certifications that help: **AWS Solutions Architect – Associate**, **CKA** (Kubernetes administrator), **HashiCorp Terraform Associate**; later AWS DevOps Engineer – Professional, CKS.

18.2 How to answer

Technical questions: the 5-part answer

1. **Definition** in one sentence.
2. **How it works** (mechanism).
3. **Example** or command.
4. **Trade-offs / when not to use it.**

5. **Your experience** ("In my project I…").

Example: *"What is a Kubernetes readiness probe?"*

> "It's a periodic check the kubelet runs to decide whether a container can receive traffic (definition). If it fails, the pod is removed from the Service's endpoints but not restarted (mechanism). For example an HTTP GET on `/ready` every 5 seconds that checks the app has loaded its config (example). It differs from liveness, which restarts the container; using a dependency check in liveness can cause cascading restarts (trade-off). In my EKS project I added readiness on `/ready` and saw zero failed requests during rolling updates (experience)."

Behavioral questions: STAR

Situation (context, briefly) → **T**ask (your responsibility) → **A**ction (what *you* did, specifically: most of the answer) → **R**esult (measurable outcome + what you learned).

Live problems: think aloud

State assumptions, ask clarifying questions, start simple, verify each step, mention what you'd do in production (tests, security, monitoring). Silence hurts more than a wrong first guess.

When you don't know

"I haven't used X directly. My understanding is … because it's similar to Y which I've used. Here's how I'd find out: …" Never bluff; interviewers drill until bluffs collapse.

18.3 Rapid-fire fundamentals with model answers

Short, correct answers to the most-asked questions. Expand each with the 5-part method when the interviewer wants depth. Each topic's notes end with more questions.

Linux

Question	Model answer
What happens when you run a command in the shell?	The shell parses it, expands variables/globs, finds the program via `PATH`, `fork()`s a child and `exec()`s the program; the parent waits and gets the exit code (`$?`)
Hard link vs soft link?	Hard link = another name for the same inode (same filesystem, survives deleting the original). Symlink = a small file pointing to a path (can cross filesystems, breaks if target removed)
Explain `chmod 750 file`	Owner rwx (7), group r-x (5), others none (0)
What is a process vs a thread? Zombie vs orphan?	Process: own memory space; threads share it. Zombie: finished, parent hasn't read its exit status. Orphan: parent died; adopted by PID 1
`kill` vs `kill -9`?	Default SIGTERM lets the process clean up; SIGKILL (9) can't be caught: use only as a last resort
What's load average?	Average number of processes running or waiting (CPU or uninterruptible I/O) over 1/5/15 min; compare with CPU count
How do you find what's using port 8080?	`ss -tulpn | grep :8080` (or `lsof -i :8080`)

Question	Model answer
What is an inode?	Metadata structure for a file (owner, permissions, size, block pointers); filenames live in directories pointing to inodes. You can run out of inodes with free space left
How does systemd manage services?	Unit files define how to start/stop/restart; `systemctl enable` creates boot links; journald collects logs (`journalctl -u`)
How do you make a script run every day at 2 AM?	Cron `0 2 * * * /path/script.sh` (or a systemd timer), with logging and absolute paths

Networking

Question	Model answer
What happens when you type a URL in a browser?	DNS resolution (cache → resolver → root → TLD → authoritative), TCP handshake to the IP on 443, TLS handshake (certificate check, keys), HTTP request, server/LB/app processing, response, rendering, more requests for assets
TCP vs UDP	TCP: connection-oriented, reliable, ordered, flow/congestion control (HTTP, SSH, DBs). UDP: connectionless, no guarantees, low overhead (DNS, video, VoIP, QUIC)
Explain the TCP three-way handshake	SYN → SYN-ACK → ACK; then data. Closing uses FIN/ACK in both directions
What is a subnet? How many hosts in a /26?	A range of IPs sharing a network prefix. /26 = 64 addresses, 62 usable hosts in classic networking (AWS reserves 5 per subnet → 59)
Public vs private IP; what is NAT?	Private ranges (10/8, 172.16/12, 192.168/16) aren't routed on the internet; NAT translates private addresses to a public one for outbound traffic
What does DNS TTL do?	How long resolvers cache a record; lower it before migrations
A record vs CNAME vs alias	A: name → IPv4. CNAME: name → another name (not at the zone apex). Route 53 alias: apex-capable pointer to AWS resources
Load balancer L4 vs L7	L4 (NLB) routes on IP/port, very fast, preserves TCP; L7 (ALB) understands HTTP: host/path routing, headers, TLS termination, WAF
What is a reverse proxy?	A server in front of backends that receives client requests and forwards them (Nginx, ALB): TLS, routing, caching, load balancing
Ports to know	22 SSH, 53 DNS, 80 HTTP, 443 HTTPS, 3306 MySQL, 5432 PostgreSQL, 6379 Redis, 6443 Kubernetes API, 9090 Prometheus

Git

Question	Model answer
`git merge` vs `git rebase`	Merge keeps history and adds a merge commit; rebase replays commits onto a new base for linear history. Never rebase shared/pushed branches
`reset` vs `revert`	Reset moves the branch pointer (rewrites history; local use). Revert creates a new commit undoing an old one (safe on shared branches)
What is `git stash`?	Temporarily shelves uncommitted changes to switch context; `git stash pop` restores them
Branching strategy you prefer?	Trunk-based development with short-lived feature branches, PR reviews, CI on every PR, feature flags; GitFlow only for products with versioned releases

Question	Model answer
How do you recover a deleted branch?	`git reflog` to find the commit, then `git branch name <sha>`

Scripting (Bash / Python)

Question	Model answer
What does `set -euo pipefail` do?	Exit on error (`-e`), error on unset variables (`-u`), fail a pipeline if any command fails (`pipefail`): safer scripts
`$@` vs `$*` ; `$?` ; `$$`	`"$@"` = all arguments as separate words (use this); `$?` = last exit code; `$$` = current PID
Why quote variables?	Unquoted variables undergo word splitting and globbing: `rm $file` with spaces deletes the wrong things. Use `"$file"`
How do you parse JSON in Bash?	`jq` , e.g. `aws ec2 describe-instances | jq -r '.Reservations[].Instances[].InstanceId'`
Python: list vs tuple vs dict vs set	List: ordered, mutable; tuple: ordered, immutable; dict: key→value; set: unique unordered items with fast membership
How do you handle errors in Python?	`try/except` for specific exceptions, `finally` /context managers for cleanup, logging, non-zero exit code for scripts
How do you call AWS from Python?	`boto3` with credentials from the role/profile (never hardcoded), paginators for large lists, handle `ClientError`

AWS

Question	Model answer
Region vs Availability Zone	Region = geographic area with multiple isolated AZs (data centers with independent power/network). Spread across AZs for HA, regions for DR/latency
Security group vs NACL	SG: instance/ENI level, stateful, allow rules only. NACL: subnet level, stateless (return traffic needs rules), allow and deny, ordered rules
Public vs private subnet	Public subnet's route table has a route to an Internet Gateway; private subnets reach the internet via a NAT gateway (outbound only)
IAM user vs role vs policy	User: long-term identity (avoid for apps). Role: assumable identity with temporary credentials (EC2, Lambda, pods, CI via OIDC, cross-account). Policy: JSON permissions document
How do you give an EC2 instance S3 access?	An IAM role with a least-privilege policy attached through an instance profile; never access keys on the instance
S3 storage classes	Standard, Intelligent-Tiering, Standard-IA, One Zone-IA, Glacier Instant/Flexible/Deep Archive; lifecycle rules move/expire objects
How do you design a highly available web app?	Multi-AZ: ALB across ≥2 AZs, Auto Scaling group (or ECS/EKS) in private subnets, RDS Multi-AZ, S3/CloudFront for static assets, Route 53 health checks, backups
RDS Multi-AZ vs read replica	Multi-AZ: synchronous standby for automatic failover (HA). Read replica: asynchronous copy for read scaling (can be promoted; also cross-region DR)
What is the shared responsibility model?	AWS secures the cloud infrastructure; you secure your data, IAM, network config, OS patches and applications

Question	Model answer
How do you control AWS costs?	Tagging, budgets and alerts, right-sizing, Savings Plans/RIs, spot for flexible workloads, auto scaling, lifecycle policies, removing idle resources, watching NAT/data transfer
What is VPC peering vs Transit Gateway?	Peering: 1:1, non-transitive. Transit Gateway: hub connecting many VPCs and on-prem with central routing
CloudWatch vs CloudTrail	CloudWatch: metrics, logs, alarms (performance/operations). CloudTrail: audit log of API calls (who did what)

Docker

Question	Model answer
Container vs virtual machine	Containers share the host kernel and isolate with namespaces/cgroups: lightweight, start in ms. VMs virtualize hardware with their own kernel: stronger isolation, heavier
Image vs container	Image: read-only layered template. Container: a running instance with a writable layer
`CMD` vs `ENTRYPOINT`	ENTRYPOINT sets the executable; CMD gives default arguments (overridable). Use exec (JSON) form so signals reach the process
How do you reduce image size?	Multi-stage builds, minimal base (slim/distroless/alpine), combine and clean package installs, `.dockerignore`
`COPY` vs `ADD`	COPY copies files; ADD also extracts archives and fetches URLs: prefer COPY
How do containers persist data?	Volumes (managed by Docker) or bind mounts; containers' writable layer is lost on removal
How do you secure containers?	Minimal pinned base images, non-root user, no secrets in images, scan (Trivy), read-only FS, drop capabilities, resource limits, signed images

Terraform

Question	Model answer
What is Terraform state and why remote state?	State maps config to real resources and tracks attributes. Remote state (S3 with locking) enables teamwork, locking and encryption; never commit state (it contains secrets)
`terraform plan` vs `apply`	Plan shows the diff between config/state/reality; apply executes it. In CI: save the plan and apply exactly that plan after approval
`count` vs `for_each`	count: index-based (removing an item shifts indexes, causing recreation). for_each: keyed by map/set (stable): preferred for collections
What are modules?	Reusable packages of resources with inputs/outputs; version them; compose environments from them
How do you handle drift?	`plan -refresh-only` to detect, then either fix the code or revert the manual change; prevent console changes with permissions and pipelines
How do you manage secrets in Terraform?	Don't put them in code; read from Secrets Manager/SSM or let services manage them (`manage_master_user_password` for RDS); mark outputs `sensitive` ; protect state
How do you import existing resources?	`import` blocks (or `terraform import`), write matching config, plan until no changes

Question	Model answer
Workspaces vs separate directories for environments	Directories (or separate state/backends per env) are clearer and safer for prod; workspaces suit identical short-lived copies

Ansible

Question	Model answer
What is Ansible and why agentless?	Configuration management over SSH (WinRM for Windows): no agent to install; push model from a control node
What is idempotency?	Running the same playbook again changes nothing if the system is already in the desired state; modules check before acting
Playbook vs role	Playbook maps hosts to tasks/roles; role is a reusable structured unit (tasks, handlers, templates, defaults, vars)
What are handlers?	Tasks run only when notified by a change, once at the end of the play (e.g. restart nginx after config change)
How do you store secrets?	Ansible Vault or lookups from a secret manager; `no_log: true` for sensitive tasks
Terraform vs Ansible	Terraform provisions infrastructure declaratively with state; Ansible configures servers/apps procedurally-declaratively without state. Commonly used together

CI/CD

Question	Model answer
CI vs continuous delivery vs continuous deployment	CI: merge often, auto build/test. Delivery: always releasable, prod deploy by decision. Deployment: every passing change goes to prod automatically
What stages does your pipeline have?	Lint → unit tests → security scans (secrets, SAST, deps) → build image → scan image → push → deploy dev → integration tests → staging → approval → prod (progressive) → verify
How do you secure a pipeline?	Least-privilege tokens, OIDC to cloud, environment protections, pinned actions, no untrusted code with secrets, scanned artifacts, signed images
Blue/green vs canary vs rolling	Blue/green: two full environments, instant switch. Canary: small % first with metric analysis. Rolling: replace instances in batches
How do you roll back?	Redeploy the previous immutable artifact (or `git revert` in GitOps); keep DB migrations backward-compatible so rollbacks are safe
How do you speed up a slow pipeline?	Caching, parallel jobs, test splitting, running only affected parts, smaller images, faster runners

Kubernetes

Question	Model answer
Control plane components	API server (front door), etcd (state), scheduler (places pods), controller manager (reconcile loops), cloud controller manager
Pod vs Deployment vs ReplicaSet	Pod: smallest unit (containers sharing network/storage). ReplicaSet keeps N pods. Deployment manages ReplicaSets for rolling updates/rollbacks
Liveness vs readiness vs startup probe	Liveness restarts a stuck container; readiness controls Service traffic; startup protects slow starts

Question	Model answer
Requests vs limits	Requests: what the scheduler reserves. Limits: ceiling; CPU throttled, memory over limit → OOMKilled
Service types	ClusterIP (internal), NodePort, LoadBalancer (cloud LB), ExternalName, headless
ConfigMap vs Secret	Both inject config; Secrets are for sensitive data (base64, not encrypted by default: use encryption at rest and external secret stores)
StatefulSet vs Deployment	StatefulSet gives stable names, ordered rollout and per-pod storage (databases, Kafka); Deployment for stateless apps
How does HPA work?	Scales replicas based on metrics (CPU % of requests, custom metrics) between min and max
How do you debug a CrashLoopBackOff?	`describe` (events, exit code), `logs --previous`, check config/secrets, probes, OOM, command
How do pods get AWS permissions on EKS?	EKS Pod Identity or IRSA: a ServiceAccount mapped to an IAM role; temporary credentials; never the node role

Monitoring and SRE

Question	Model answer
Four golden signals	Latency, traffic, errors, saturation
SLI vs SLO vs SLA	SLI: measured ratio (good/valid events). SLO: internal target (99.9% over 30 days). SLA: contractual promise with penalties, looser than the SLO
Error budget for 99.9% per 30 days	0.1% → 43.2 minutes of full downtime (or 0.1% of requests failing)
What should page someone?	Symptoms affecting users now (SLO burn, outage), always actionable with a runbook; everything else is a ticket
Prometheus counter queries	Use `rate()` over a window, then aggregate with `sum by (...)`; percentiles with `histogram_quantile` on bucket rates
What is a blameless postmortem?	A structured review of an incident focusing on systemic causes and action items, not on blaming individuals

Security and GitOps

Question	Model answer
How do you avoid secrets in code?	Secret managers + IAM roles/OIDC, pre-commit and push-protection scanning, ExternalSecrets in Kubernetes, rotation
What is shift-left security?	Moving security checks early: IDE, pre-commit, PR (SAST, SCA, IaC scanning) where fixes are cheapest
What is least privilege?	Granting only the minimum permissions needed, scoped to specific resources and time
What is GitOps?	Git as the single source of truth for desired state; an in-cluster agent (Argo CD/Flux) pulls and continuously reconciles, giving audit trail, drift correction and easy rollback
Argo CD sync vs health	Sync: does live state match Git? Health: are the resources actually working?

18.4 Scenario questions with model answers

1. "Production is down after a deployment. What do you do?"

Confirm impact (dashboards, error rate, which services). Communicate: declare an incident, open a channel. Mitigate first: roll back to the previous version (redeploy the previous image or `git revert` in GitOps) since the timing points to the deploy. Verify recovery with the same metrics. Then investigate the diff (code, config, migrations), write a postmortem, and add the missing test or canary analysis that would have caught it.

2. "An application is slow. How do you troubleshoot?"

Define: which endpoints, which users, since when. Check golden signals: latency percentiles, errors, traffic, saturation. Use traces to find the slow span. If it's the DB: slow query logs, locks, connection pool. If it's resources: CPU throttling, memory/GC, disk I/O. Check what changed (deploys, traffic, data volume). Fix, verify on p95/p99, add an alert/SLO if it wasn't caught early.

3. "Your AWS bill doubled. What do you do?"

Cost Explorer grouped by service, then usage type and tags, compared with last month. Typical culprits: NAT gateway data processing, inter-AZ/inter-region transfer, forgotten instances/EBS volumes/snapshots/load balancers, log ingestion, an autoscaler that didn't scale down, or a leaked key used for mining (check CloudTrail, unknown regions). Fix, then add budgets with alerts, tagging policies, and anomaly detection.

4. "A developer needs access to production logs but not to change anything."

Give a read-only role through SSO scoped to the log groups/namespaces they need (CloudWatch Logs read, or Grafana/Loki viewer role, or Kubernetes `view` role in the namespace without `secrets` and `pods/exec`), time-bound if possible, audited via CloudTrail.

5. "How would you migrate an app from EC2 to Kubernetes?"

Containerize (Dockerfile, config via env, logs to stdout, health endpoints, graceful shutdown). Write manifests/Helm chart with probes, resources, HPA, PDB. Externalize state (RDS/S3). Set up CI to build/scan/push and GitOps to deploy. Run in staging, load test, then shift traffic gradually (weighted DNS or ALB) with monitoring, keeping EC2 as a rollback until stable.

6. "Terraform plan shows it will destroy and recreate the production database."

Stop. Read which argument forces replacement. If it's a rename, use a `moved` block; if an immutable setting changed, revert it or plan a controlled migration (snapshot, maintenance window). Add `prevent_destroy` and deletion protection on critical resources; require plan review in CI.

7. "A secret was committed to GitHub."

Rotate/revoke it immediately, check usage logs for abuse, clean up, then remove from history if needed, and prevent recurrence with pre-commit secret scanning, push protection and moving to OIDC/roles.

8. "Kubernetes pods keep getting OOMKilled."

Confirm via `describe` (Last State: OOMKilled, exit 137). Compare memory usage to the limit over time: a leak (steady growth) vs undersized limit (spikes at load). Check runtime heap settings (JVM/Node). Fix the leak or right-size requests/limits using VPA recommendations; add an alert on memory vs limit.

9. "Disk is filling up on a server every week."

> Find what grows (`du`), usually logs or temp files. Configure logrotate/journal limits, application log levels, ship logs centrally, clean temp files with a timer; alert with `predict_linear`. If data legitimately grows, move it to S3 or grow the volume and plan capacity.

10. "How do you do a zero-downtime deployment?"

> Rolling or blue/green with readiness probes, graceful shutdown (SIGTERM handling, preStop delay, connection draining), `maxUnavailable: 0`, backward-compatible DB migrations (expand/contract), enough capacity (PDBs), and automated verification with rollback.

18.5 System design for DevOps interviews

The framework (spend the first minutes here)

1. **Clarify requirements:** users/traffic (requests/s, peak), latency, availability target (SLO), data size and consistency, regions, compliance, budget, team size.
2. **High-level architecture:** draw the request path and main components.
3. **Deep dive:** compute, data, networking, security, CI/CD, observability.
4. **Reliability:** failure modes (instance, AZ, region, dependency), scaling, backups and DR (RPO/RTO).
5. **Security and cost:** IAM, encryption, network isolation, secrets; cost drivers and optimizations.
6. **Trade-offs and evolution:** what you'd do first, what later, what you'd revisit.

Design 1: highly available web application on AWS

```
Users → Route 53 → CloudFront (+ WAF, static assets from S3)
                          |
                          ▼
               ALB (public subnets, 3 AZs, ACM TLS)
                          |
        ECS Fargate / EKS / ASG (private app subnets, 3 AZs, auto scaling on CPU/RPS)
                |                          |
    RDS PostgreSQL Multi-AZ        ElastiCache Redis (cluster mode, Multi-AZ)
    (private data subnets,         sessions / cache
     automated backups, PITR,
     read replica for reads)
 Secrets Manager · KMS · CloudWatch/Prometheus · CloudTrail · GuardDuty · AWS Backup
 Terraform for everything; CI/CD with image scanning; blue/green or canary deploys
```

Talking points: stateless app tier (sessions in Redis), Multi-AZ everywhere, health checks at every layer, autoscaling with headroom to survive losing one AZ, NAT gateway per AZ (availability vs cost), VPC endpoints for S3/ECR to cut NAT traffic, DR: backups copied cross-region (RPO hours) or warm standby in a second region (RPO minutes, higher cost).

Design 2: CI/CD platform for 50 microservices

- Monorepo or polyrepo with a **reusable workflow** (test, scan, build, sign, push) versioned centrally; each service calls it.
- Ephemeral, autoscaled runners (GitHub-hosted or self-hosted on Kubernetes with ARC), OIDC to AWS, per-environment roles.
- Artifacts: ECR with immutable tags, scan on push, lifecycle policies; SBOM and signatures.
- Deployment: GitOps config repo, Argo CD ApplicationSets per environment, promotion by PR, Argo Rollouts canary with Prometheus analysis for critical services.

- Governance: branch protection, CODEOWNERS, required checks, Kyverno verifying signatures, audit trail in Git.
- Metrics: DORA dashboards, pipeline duration and flakiness, failed deploy rate.

Design 3: observability for an EKS platform

- Metrics: kube-prometheus-stack per cluster (two replicas) → remote write to Amazon Managed Prometheus or Mimir; ServiceMonitors via golden-path templates.
- Logs: Fluent Bit/Alloy DaemonSet → Loki on S3 (or CloudWatch Logs) with retention tiers; JSON logs with trace IDs.
- Traces: OpenTelemetry SDK/auto-instrumentation → OTel Collector (tail sampling) → Tempo/X-Ray.
- Grafana with SSO and team folders; dashboards as code.
- Alerting: SLO burn-rate alerts per service, platform alerts (nodes, control plane, certificates), Alertmanager → PagerDuty with routing by team label; dead man's switch.
- Cost control: drop high-cardinality metrics, sample traces, log retention by class.

Design 4: disaster recovery strategy

Strategy	RPO / RTO	Cost	How
Backup and restore	Hours / hours–day	Lowest	Cross-region backups (AWS Backup, snapshots), IaC to rebuild
Pilot light	Minutes / tens of minutes	Low	Data replicated (cross-region read replica, S3 replication), minimal core running, scale up on failover
Warm standby	Seconds–minutes / minutes	Medium	Scaled-down full copy running in the second region
Multi-site active/active	Near zero / near zero	Highest	Both regions serve traffic (Route 53 latency/failover, global databases like Aurora Global/DynamoDB global tables)

Always: define RPO/RTO with the business, automate the failover runbook, **test it regularly** (game days).

18.6 Hands-on tasks you may get (with solutions)

Task 1: top 5 client IPs in an Nginx access log

```
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -5
```

Follow-ups: count 5xx responses per minute (`awk '$9 ~ /^5/ {print substr($4, 2, 17)}' access.log | uniq -c`), handle rotated `.gz` logs (`zcat -f access.log*`).

Task 2: disk usage alert script

```
#!/usr/bin/env bash
# disk-alert.sh: warn when any filesystem is above a threshold (default 80%)
set -euo pipefail

threshold="${1:-80}"
if ! [[ "$threshold" =~ ^[0-9]+$ ]] || (( threshold < 1 || threshold > 99 )); then
  echo "usage: $0 [threshold 1-99]" >&2
  exit 2
fi

status=0
while read -r usage mount; do
  pct="${usage%\%}"
  if (( pct >= threshold )); then
    echo "WARNING: ${mount} is at ${pct}% (threshold ${threshold}%)"
    status=1
  fi
done < <(df -P -x tmpfs -x devtmpfs | awk 'NR > 1 {print $5, $6}')

if (( status == 0 )); then
  echo "OK: all filesystems below ${threshold}%"
fi
exit "$status"
```

Exit code 1 when any filesystem is over the threshold, so cron/monitoring can act on it.

Task 3: check a list of URLs

```
#!/usr/bin/env bash
# check-urls.sh: print HTTP status and total time for each URL in a file
set -euo pipefail

file="${1:?usage: $0 urls.txt}"
failed=0
while IFS= read -r url; do
  [[ -z "$url" || "$url" == \#* ]] && continue
  result=$(curl -s -o /dev/null -m 10 -w '%{http_code} %{time_total}' "$url" || true)
  code="${result%% *}"
  if [[ "$code" =~ ^(2|3)[0-9][0-9]$ ]]; then
    echo "OK   ${result} ${url}"
  else
    echo "FAIL ${result:-000} ${url}"
    failed=$((failed + 1))
  fi
done < "$file"
echo "failed: ${failed}"
(( failed == 0 ))
```

Task 4: summarize HTTP status codes from a log with Python

```
#!/usr/bin/env python3
"""Count HTTP status codes in an Nginx/Apache combined log and print JSON."""
import json
import re
import sys
from collections import Counter

# 1.2.3.4 - - [18/Oct/2026:10:00:00 +0530] "GET /x HTTP/1.1" 200 512 "-" "curl/8"
LINE = re.compile(r'^(\S+) \S+ \S+ \[[^\]]+\] "[^"]*" (\d{3}) ')


def summarize(lines):
    codes = Counter()
    ips = Counter()
    bad = 0
    for line in lines:
        match = LINE.match(line)
        if not match:
            bad += 1
            continue
        ip, status = match.groups()
        codes[status] += 1
        ips[ip] += 1
    total = sum(codes.values())
    errors = sum(n for code, n in codes.items() if code.startswith("5"))
    return {
        "total": total,
        "by_status": dict(sorted(codes.items())),
        "error_rate_5xx": round(errors / total, 4) if total else 0.0,
        "top_ips": ips.most_common(3),
        "unparsed_lines": bad,
    }


if __name__ == "__main__":
    path = sys.argv[1] if len(sys.argv) > 1 else "/var/log/nginx/access.log"
    with open(path, encoding="utf-8", errors="replace") as handle:
        print(json.dumps(summarize(handle), indent=2))
```

Task 5: find EC2 instances missing an `owner` tag (boto3)

```
#!/usr/bin/env python3
"""List EC2 instances without an 'owner' tag in one region (read-only)."""
import sys

import boto3
from botocore.exceptions import BotoCoreError, ClientError


def untagged_instances(region: str, tag_key: str = "owner"):
    ec2 = boto3.client("ec2", region_name=region)
    paginator = ec2.get_paginator("describe_instances")
    for page in paginator.paginate():
        for reservation in page["Reservations"]:
            for inst in reservation["Instances"]:
                tags = {t["Key"]: t["Value"] for t in inst.get("Tags", [])}
                if tag_key not in tags:
                    yield inst["InstanceId"], inst["State"]["Name"], tags.get("Name", "-")


if __name__ == "__main__":
    region = sys.argv[1] if len(sys.argv) > 1 else "ap-south-1"
    try:
        for instance_id, state, name in untagged_instances(region):
            print(f"{instance_id}\t{state}\t{name}")
    except (BotoCoreError, ClientError) as err:
        print(f"error: {err}", file=sys.stderr)
        sys.exit(1)
```

Task 6: fix this Dockerfile

```
# BEFORE: what's wrong?
FROM python:latest
COPY . /app
RUN pip install -r /app/requirements.txt
ENV DB_PASSWORD=supersecret
CMD python /app/app.py
```

Problems: unpinned `latest` base (unreproducible), huge full image, `COPY .` before dependency install (no layer caching, copies `.git` and secrets), secret baked into the image, runs as root, shell-form `CMD` (signals not delivered), no health check.

```
# AFTER
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
RUN useradd --uid 10001 --no-create-home appuser
USER 10001
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s \
  CMD ["python", "-c", "import urllib.request as u; u.urlopen('http://localhost:8000/healthz')"]
CMD ["python", "-m", "src.app"]
```

The password comes from the runtime environment (secret manager → env var or mounted file), plus a `.dockerignore` excluding `.git`, `.env`, tests and caches.

Task 7: why doesn't this Service get traffic?

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: frontend          # BUG 1: matches no pods (pods are labeled app=web)
  ports:
    - port: 80
      targetPort: 8080     # BUG 2: nginx listens on 80
```

Fix: `selector: {app: web}` and `targetPort: 80`. Then also mention what's missing for production: resources, probes, securityContext, PDB. Diagnose live with `kubectl get endpointslices -l kubernetes.io/service-name=web` (empty).

Task 8: Terraform an S3 bucket the secure way

```
resource "aws_s3_bucket" "artifacts" {
  bucket = "acme-artifacts-${var.env}"
  tags = {
    owner = "platform"
    env   = var.env
  }
}

resource "aws_s3_bucket_versioning" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id
  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
    }
    bucket_key_enabled = true
  }
}

resource "aws_s3_bucket_public_access_block" "artifacts" {
  bucket                  = aws_s3_bucket.artifacts.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

variable "env" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.env)
    error_message = "env must be dev, staging or prod."
  }
}
```

18.7 Behavioral questions (STAR)

Prepare 6–8 stories from your projects, labs, studies or previous jobs; each story can answer several questions.

Question	What they want
Tell me about yourself	60–90 seconds: background → why DevOps → what you built (projects) → what you want next
A time you solved a hard technical problem	Method, persistence, learning
A failure or mistake you made	Ownership, honesty, what you changed afterwards
A conflict with a teammate / developer	Empathy, data-driven discussion, outcome
When you automated something	Initiative, impact in time/errors saved
How you learn new technology	Concrete approach: docs, labs, building projects, teaching others

Question	What they want
Working under pressure (incident)	Calm, structured, communication
Why this company / role?	You researched them: their stack, product, engineering blog

Example STAR (from this program's final project):

> **S:** In my final project I deployed a microservice app on EKS, and every deployment caused a burst of 502 errors. **T:** I owned the delivery pipeline and wanted zero-downtime releases. **A:** I reproduced it with a load test during a rollout, saw pods were killed while the ALB still sent traffic, then added a readiness probe, a `preStop` sleep longer than the target deregistration delay, SIGTERM handling in the app, and `maxUnavailable: 0`. I added an alert on ALB 5xx during deployments. **R:** Rollouts now complete with zero failed requests under load; I documented it as a checklist in the repo's README.

18.8 Questions to ask the interviewer

- What does a typical week look like for this role? How much is project work vs operations?
- How are deployments done today, and how often? What's the biggest pain in the delivery process?
- How does on-call work, and how many pages does a typical shift get?
- How do you measure reliability (SLOs)? How are incidents reviewed?
- What would success look like in the first 90 days?
- How does the team learn (certifications budget, mentoring, internal tech talks)?

18.9 Job search strategy

Target roles	Focus
Junior DevOps / Cloud Engineer	Linux, AWS core, Terraform, Docker, CI/CD, scripting
Cloud Support / Operations Engineer	Strong troubleshooting, networking, AWS services: a common entry door
Site Reliability Engineer (junior)	Linux, monitoring, incidents, scripting, Kubernetes
Platform Engineer (later)	Kubernetes, GitOps, IaC at scale, developer experience

- Apply with a **tailored resume** per job description (mirror their stack where you truly have it).
- Referrals beat cold applications: engage with engineers on LinkedIn, local meetups (AWS User Groups, CNCF/Kubernetes community events), and open-source contributions (documentation fixes count).
- Track applications in a spreadsheet (company, role, date, stage, follow-up).
- After each interview, write down every question you got and your gaps; turn them into labs.
- Research the salary range for the role and city from several sources before the recruiter call; state a researched range, and negotiate on the full package (base, bonus, learning budget, remote policy).

18.10 Final two-week revision plan

Days	Focus
1–2	Linux and networking: rapid-fire tables, 60-second checklist, subnetting drills
3	Git, Bash and Python: write tasks 1–5 from memory

Days	Focus
4–5	AWS: VPC design drawing from memory, IAM evaluation, HA architecture, costs
6	Docker and Terraform: Dockerfile and S3/VPC Terraform from memory
7	Ansible and CI/CD: pipeline stages, deployment strategies, security
8–9	Kubernetes: write all core manifests from memory; troubleshooting tree
10	Monitoring/SRE and DevSecOps: PromQL library, SLO math, leaked-secret playbook
11	GitOps and platform engineering: Argo CD flow, ApplicationSets, golden paths
12	System design: practice designs 1–4 out loud, 30 minutes each, on paper
13	Behavioral: rehearse 6–8 STAR stories; record yourself
14	Mock interview end to end; review weak spots; rest

Final checklist

- [ ] Resume with measurable project bullets; GitHub repos with READMEs and diagrams; no secrets anywhere.
- [ ] I can explain every project end to end, including why I chose each tool and what I'd improve.
- [ ] I can answer every rapid-fire question in 18.3 in under a minute.
- [ ] I can complete tasks 1–8 in 18.6 without notes.
- [ ] I can whiteboard designs 1–4 with trade-offs, security and cost.
- [ ] I have STAR stories ready and questions to ask the interviewer.

REMEMBER

- Behavioral questions: **STAR** = Situation, Task, Action, Result.
- "I don't know, but here is how I would find out" is a good answer.